



MULTIMEDIA-PROGRAMAZIOA ETA GAILU MUGIKORRAK ANDROID - ACTIVITY

1. AURKIBIDEA

1. Zer da Activity bat?
2. Zer da Layout bat?
3. Dependentsiak gehitu
4. Activity baten zatiak Android-en
5. Zer da Activity bat Android-en?
6. Android manifest
7. Programazio kontzeptuak
8. Programazioa
9. Main_Activity
10. Activity baten bizi-zikloa

2. ZER DA ACTIVITY BAT?

Gure lehen jarduera, **MainActivity**.

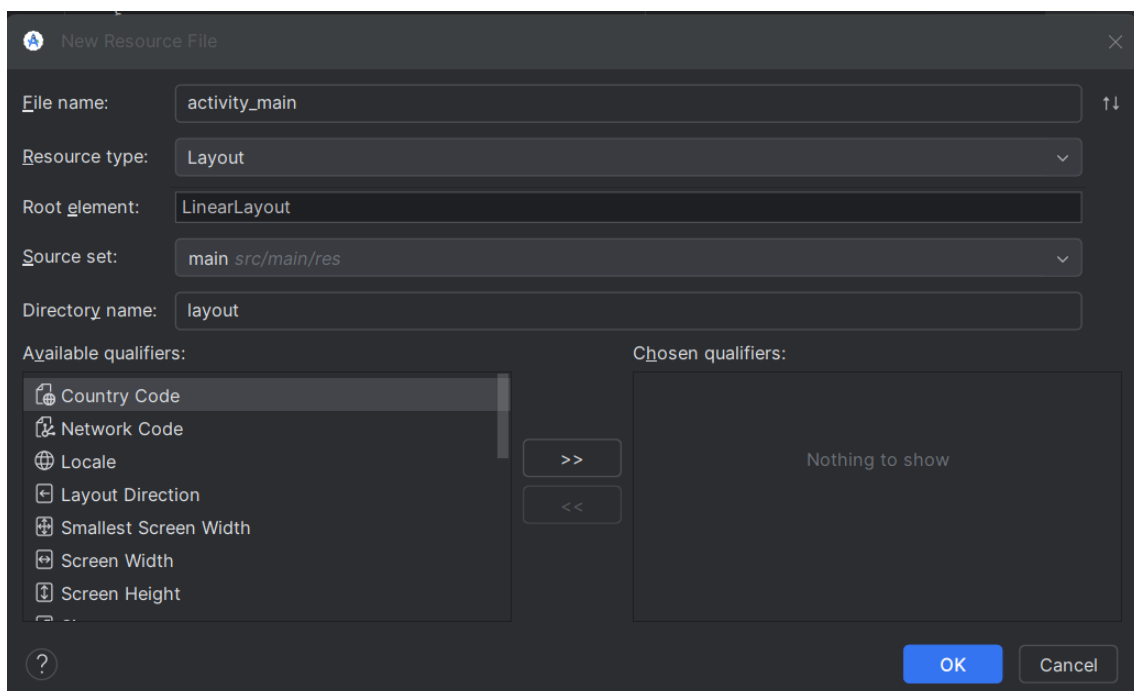
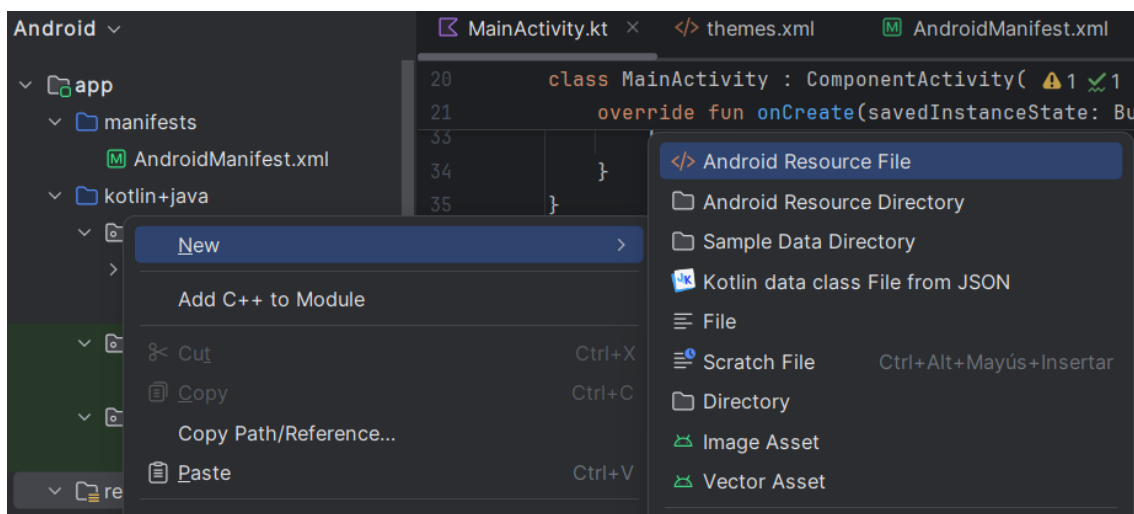
- **Activity** bat erabiltzaileak ikusten duen pantaila bakoitza da, eta bertan adierazten dira metodoak (egin daitekeena).
- **Activity**-ek bi zati dituzte, logikoa eta ikusizkoa (layout-ak)
 - **Zati logikoan**, Kotlinen garatuko duguna, erabiltzailearen interakzio bakoitzari erantzuna emango diogu.

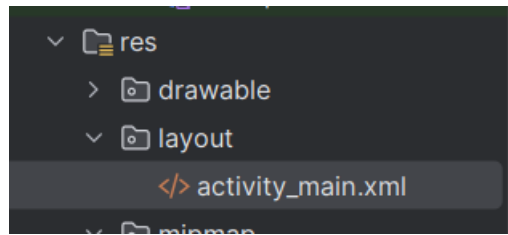
- **Activity**, funtsean, **AppCompatActivity** zabaltzen duen kotlin (.kt) klasea da, Androidekin komunikatzeko behar diren metodoak izango ditu. Beraz, beti izango dugu **OnCreate** funtzioa, gure jarduera sortzeko eta layout bat esleitzeko ardura izango duena.

3. ZER DA LAYOUT BAT?

Proiektuan layout bat gehitzeko:

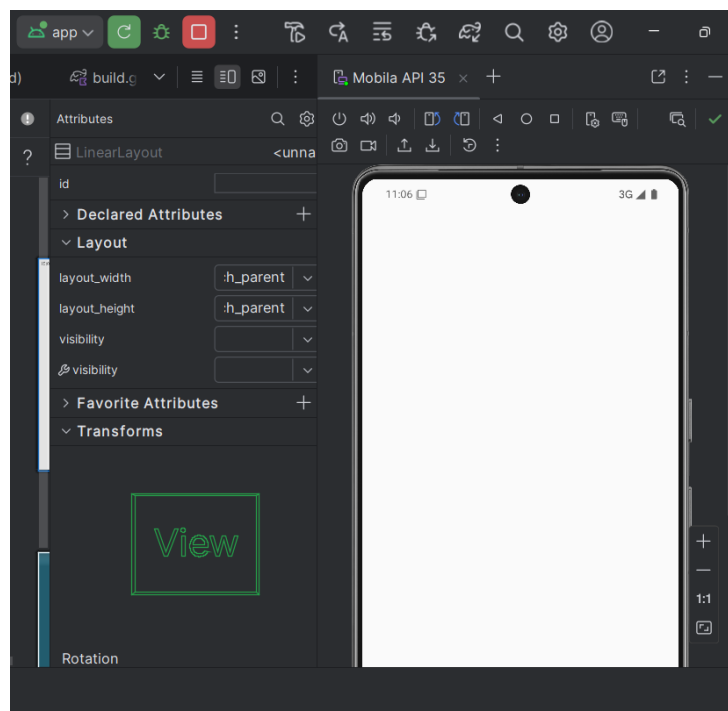
res→new→ Layout Resource File





```
1 Usage
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContentView(R.layout.activity_main)
    }
}
```

- **Layout**-a jardueraren bigarren zatia da, interfazea.
- Bertan, osagaiak gehituko dira, hala nola botoiak, irudiak eta abar.
- Fitxategi horiek oso sinpleak dira, eta osagaiak eramanetz edo XMLn gehituz osa daitezke.

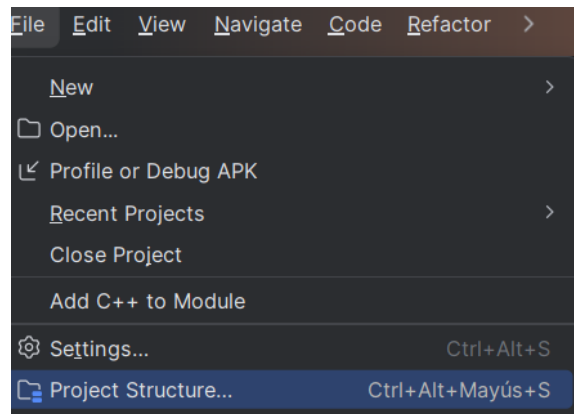


4. DEPENDENTZIAK GEHITU

Project Structure erabiltzea ikusgarriagoa da eta bertsio akatsak saihesten ditu. Hona hemen nola gehitu adibidez **ConstraintLayout** pausoz pauso:

Project Structure irekitzeko

Android Studio-n joan **File** → **Project Structure...** Leiho bat irekiko da hainbat fitxekin: **Modules, Dependencies, SDK Location...**

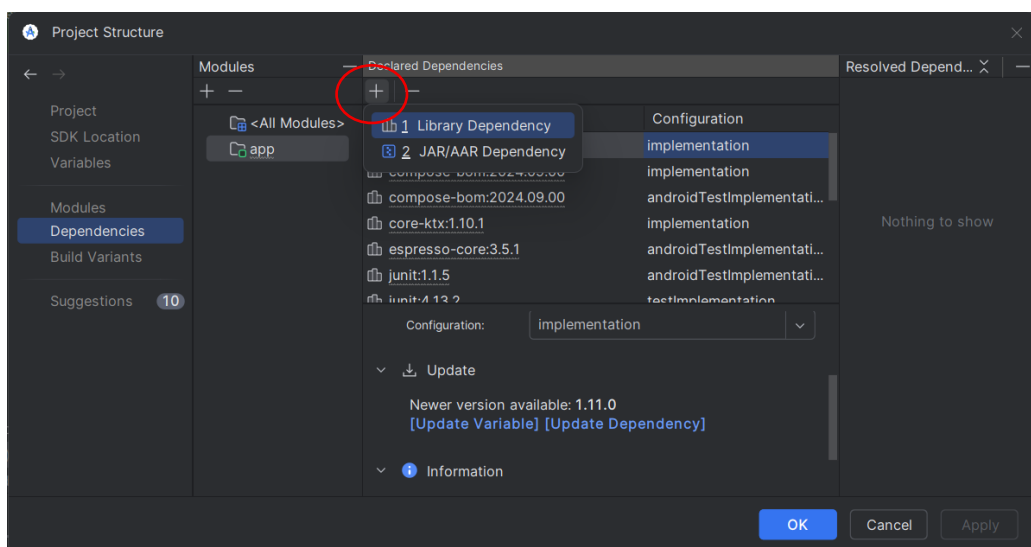


Moduloa hautatzea

Ezkerreko zutabean, hautatu zure modulua (normalean app). Klik egin **Dependencies** fitxan.

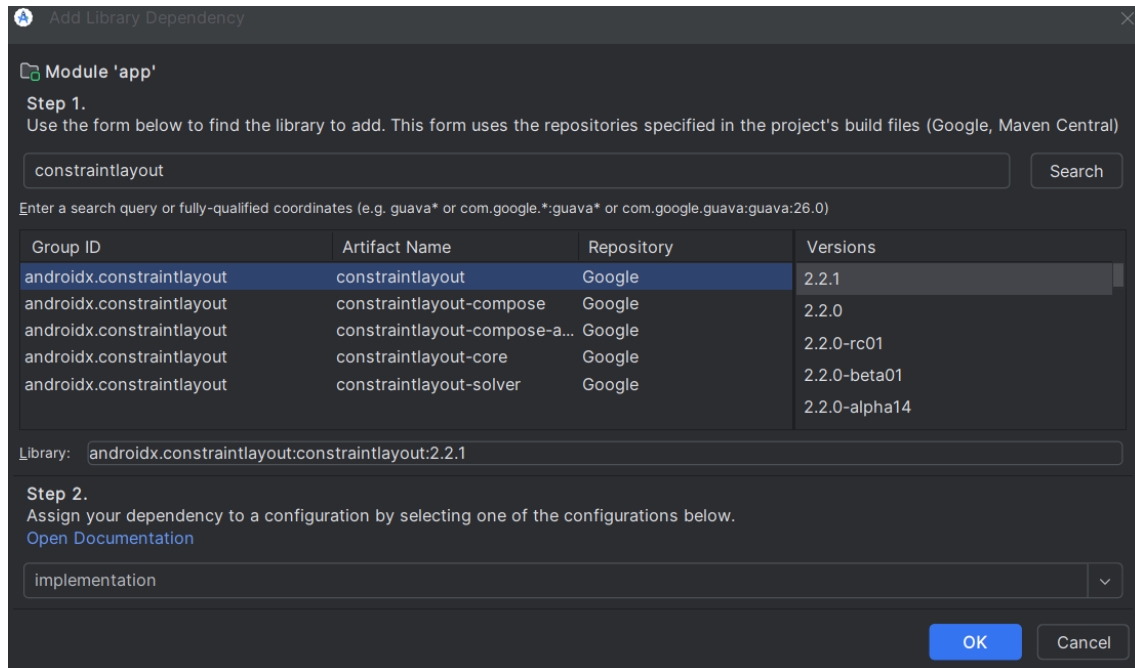
Dependentzia bat gehitzea

Klik egin **+** botoian (goiko eskuineko izkina). Hautatu **Library dependency**.



ConstraintLayout bilatzea

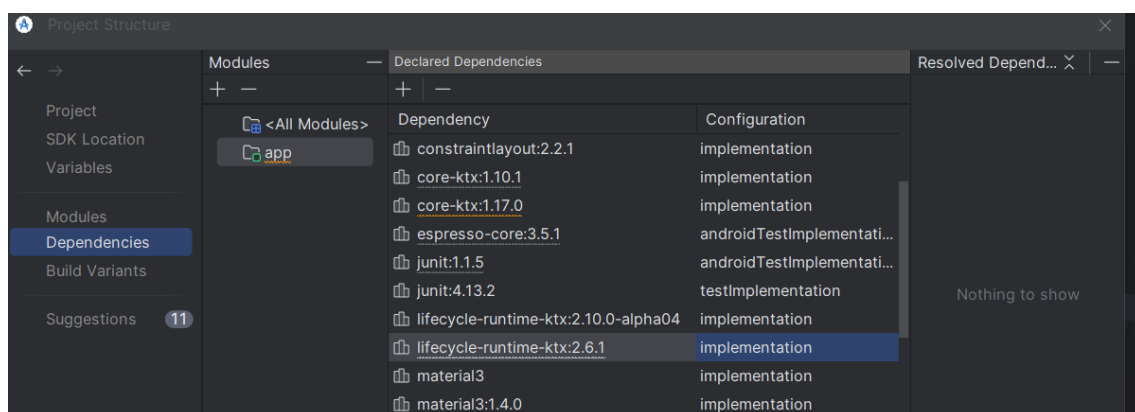
Leihoan, idatzi **constraintlayout**. Zenbait bertsio agertuko dira, adibidez:



Hautatu nahi duzun bertsioa eta klik egin **OK**.

Berdina egin

- core-ktx
- lifecycle-runtime-ktx



Sinkronizatzea

Android Studio-k automatikoki gehituko du lerroa zure build.gradle fitxan eta **Sync Now** eskatuko dizu. Klik egin eta prest.

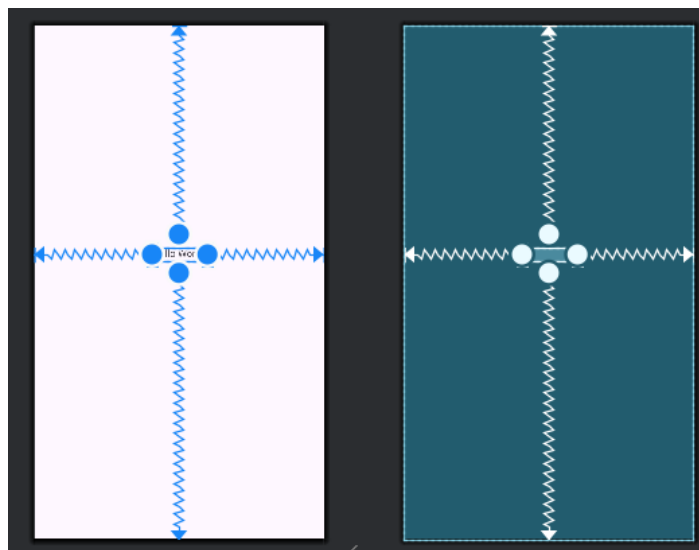
```
Starting Gradle Daemon...
Gradle Daemon started in 5 s 784 ms
> Task :prepareKotlinBuildScriptModel UP-TO-DATE

BUILD SUCCESSFUL in 52s
```

5. ACTIVITY BATEN ZATIAK ANDROID-EN

Proiektua sortzen denean, lehenik eta behin, jarduera(activity) bat sortzen da, **MainActivity**. MainActivity hori bi zati izango ditu: **klase** bat eta **xml** fitxategi bat.

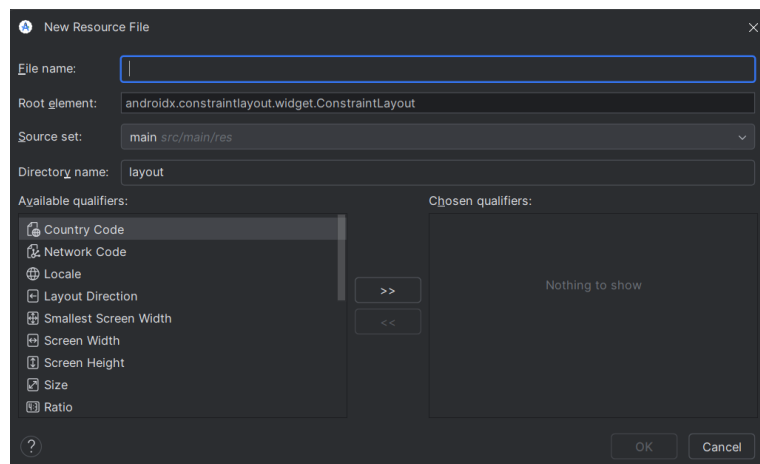
```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContentView(R.layout.activity_main)
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main)) { v, insets ->
            val systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars())
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom)
            insets
        }
    }
}
```



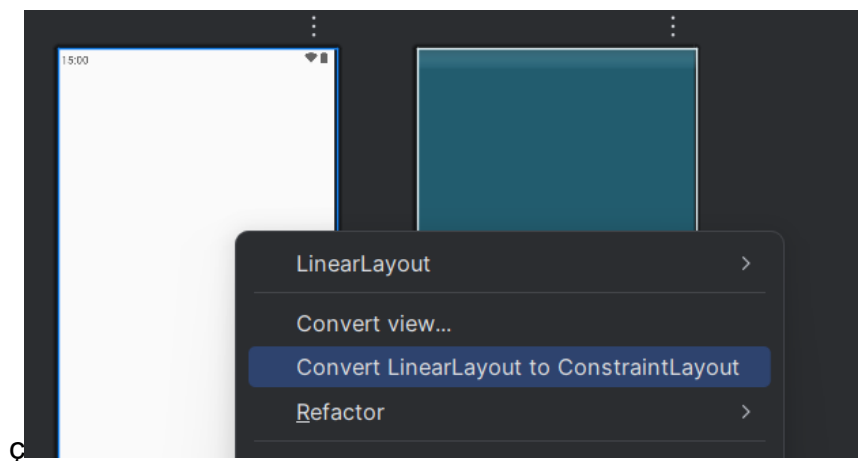
```
1 <?xml version="1.0" encoding="utf-8"?>
2 <androidx.constraintlayout.widget.ConstraintLayout xmlns:android="
3     xmlns:app="http://schemas.android.com/apk/res-auto"
4     xmlns:tools="http://schemas.android.com/tools"
5     android:id="@+id/main"
6     android:layout_width="match_parent"
7     android:layout_height="match_parent"
8     tools:context=".MainActivity">
9
10    <TextView
11        android:layout_width="wrap_content"
12        android:layout_height="wrap_content"
13        android:text="Hello World!"
14        app:layout_constraintBottom_toBottomOf="parent"
15        app:layout_constraintEnd_toEndOf="parent"
16        app:layout_constraintStart_toStartOf="parent"
17        app:layout_constraintTop_toTopOf="parent"
18        app:layout_constraintVertical_bias="0.442" />
19
20    </androidx.constraintlayout.widget.ConstraintLayout>
```

Kontuz gehitzen dan layout motarekin!

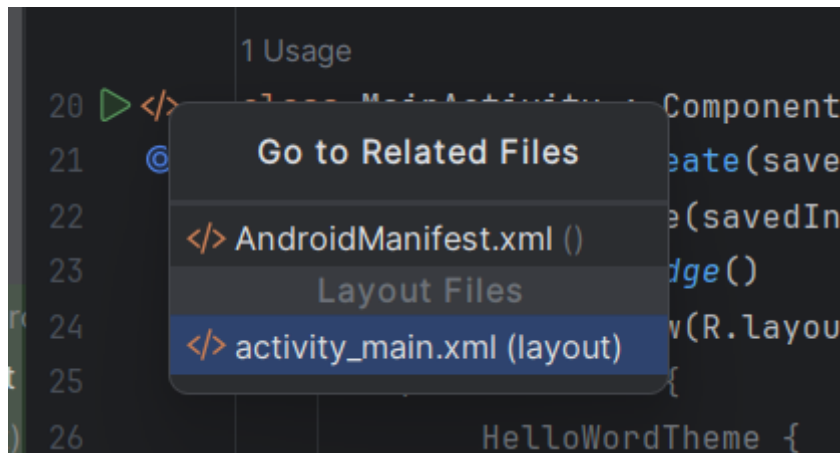
androidx.constraintlayout.widget.ConstraintLayout



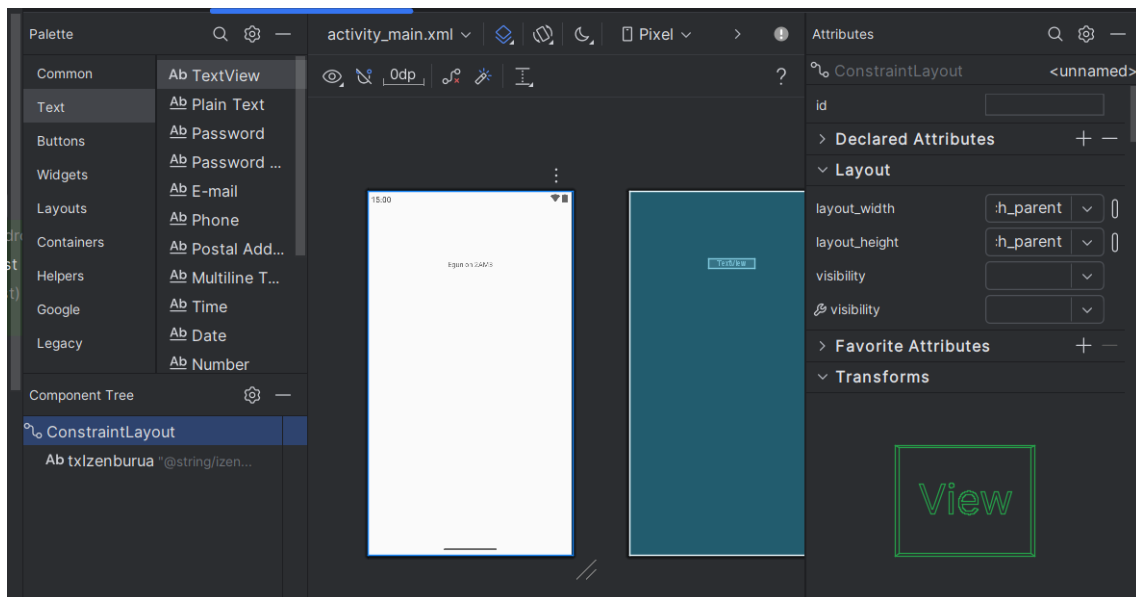
LinearLayout gehitzen bada beti ein al da transformazioa ConstarintLayout-era.



MainActivity-n egonda </> irudiari klikatu eskero xml fitxategietara joako aukera emango digu.



Irudi honetan, pantailaren osagaiak baino ez ditugu; oraingoz, **textView** hau baino ez dugu. Objeto batzuk ditugu, pantailara eraman ditzazkegunak, hala nola **testuak**, **botoiak**, **irudiak**, eta, gero, haiekin gauzak egiten joan gaitezke.



6. ANDROID MANIFEST

Android Manifest, gure aplikazioak egiten duenaren maila altuko konfigurazioa da eta honelako gauzak defini daitezke:

- **label**: aplikazioaren izena identifikatzen dugu
- **icon**: erabiliko den ikonoa identifikatzen dugu, defektuz sortzen da besterik adierazi ezean
- **theme**: gure aplikazioari itxura ematen dion gaia.

```
<?xml version="1.0" encoding="utf-8"?>
```

Fitxategia XML formatuan dagoela adierazten du.

(encoding="utf-8" esan nahi du testuak UTF-8 kodeketan gordetzen direla, hau da, karaktere guztiak ondo erakusteko moduan.)

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
  xmlns:tools="http://schemas.android.com/tools">
```

Fitxategiaren **erro elementua** da: *manifest*.

- **xmlns:Android**: Android-eko etiketen estandarra identifikatzeko da.
- **xmlns:tools**: garapen-tresnek (Android Studio bezalakoek) erabiltzen duten espazioa da; debug edo konfigurazio berezietarako.

```
<application
  android:allowBackup="true"
  android:dataExtractionRules="@xml/data_extraction_rules"
  android:fullBackupContent="@xml/backup_rules"
  android:icon="@mipmap/ic_launcher"
  android:label="@string/app_name"
  android:roundIcon="@mipmap/ic_launcher_round"
  android:supportsRtl="true"
  android:theme="@style/Theme.HelloWord">
```

Hau da **aplikazioaren konfigurazio nagusia**.

- **android:allowBackup="true"**: erabiltzaileak aplikazioaren datuak gorde eta berreskuratu ahal izango ditu (backup).
- **android:dataExtractionRules** eta **android:fullBackupContent**: backup edo datuen esportazioaren arauak definitzen dituzten XML fitxategiak.
- **android:icon**: aplikazioaren ikono nagusia.
- **android:label**: aplikazioaren izena (testu kate batetik hartua: **@string/app_name**).
- **android:roundIcon**: ikono biribildua, Android bertsio berrietan erabiltzen dena.
- **android:supportsRtl="true"**: hizkuntza eskuinetik ezkerrera (RTL, adib. arabiera) dutenentzat interfazea egokitzea.
- **android:theme**: aplikazioaren itxura edo *style*-a zehazten du.

```
<activity
    android:name=".MainActivity"
    android:exported="true"
    android:label="@string/app_name"
    android:theme="@style/Theme.HelloWorld">
```

Hemen deklaritzen da **MainActivity** (zure aplikazioaren pantaila nagusia).

- **android:name=".MainActivity"**: klasearen izena (Activity nagusia).
- **android:exported="true"**: Activity hau kanpoko beste aplikazioek abiarazi dezaketen ala ez zehazten du. Android 12tik aurrera derrigorrezkoa da.
- **android:label**: pantaila honen izena.
- **android:theme**: Activity horren itxura zehazten du.

```
<intent-filter>
    <action android:name="android.intent.action.MAIN" />
    <category android:name="android.intent.category.LAUNCHER" />
</intent-filter>
```

Activity hau **abiarazteko nagusia** dela adierazten du:

- **action.MAIN**: aplikazioaren hasierako sarrera-puntua dela.
- **category.LAUNCHER**: pantaila honetan sortuko dela aplikazioaren ikonoa, erabiltzaileak klik egiten duenean has dadin.

Adierazi behar diogu zein pantaila erabiliko diren proiektuan, jarduerak(activity). Jarduera hemen gehitzen ez baditugu, ez dugu modurik izango gure proiektuan erabiltzeko. Jarduera Nagusia zein den ere adierazi beharko da, abiarazteko botoiari ematean bera izango baita lehenengo ikusiko dena.

7. PROGRAMAZIO KONTZEPTUAK

- **Klase** bat elementu baten irudikapena izango da.
- Klase batek bi gauza izan ditzake: **funtzioak** (ekintza bat gauzatzeko kode-zatiak) edo **egoera** (klase horrek gorde ditzakeen balioak dira, gero haiekin gauzak egiteko).
- Klase bakarretik **objektu** kopuru mugagabea sor daiteke. Normalean, objektu horiei klase baten **instantzia** espezifikoa deitzen zaie, eta beraz, objektuek egoera desberdinak izan ditzakete.

Adibidez: **persona klase** mota bat dut, eta pertsona klase horrek bi atributo izango ditu: **izen** eta **adin** bat.

Guk hainbat objektu izan genitzake, esaterako Jon izena eta 20 urte duen pertsona bat, eta beste bat Tom izena eta 15 urte duena. Bi pertsona ditugu, bi objektu, pertsona klaseko horretako bi instantzia, eta pertsona bakoitzak balio desberdina du, hau da, egoera desberdina. Beraz, egoera desberdinak dituzten objektu horiekin eragiketak egiten ditugunean, eragiketek emaitza desberdinak itzul ditzakete.

```

4 Usages
class Persona // ♦ Eraikitzailea (constructor)
    (// ♦ Getter eta setter metodoak
    // ♦ Atributuak (propiedades)
    var izena: String?, var adin: Int
    )
2 Usages
val o1 =Persona( izena = "Jon",  adin = 23);
val o2 =Persona( izena = "Ane",  adin = 15);

```

Aldagai bat aldi baterako balio bat gordetzen duen elementu bat da. Adibidez, guk **o1** eta **o2** aldagaiak sortu ditugu. Ondoren, aldagai horiek erabil daitezke, adibidez, funtzio batean, **o1.izena** eta **o1.adin** egin dezakegu izena eta adina berreskuratzeke, objektu bat sortzeko, bi balio gehitzeko, edo beste eragiketa bat egiteko eta emaitza aldagai horretan gordetzeko.

```

// ♦ Metodo gehigarri bat (adibidez)
fun agurtu(context: Context?) {
    Toast.makeText(
        context,
        text = "Kaixo, " + o1.izena + "! Zure adina da " + o1.adin + " urte.",
        duration = Toast.LENGTH_LONG
    ).show()
}

```

Funtzioa: funtzio horren izenari deituz exekutatu ditzakegun kode-multzoak dira. Horrela, gauza konplexuagoak funtzio bakar batean gorde ditzakegu, eta, gainera, hainbat tokitatik dei diezaiokegu, eta kodearen zati hori berrerabil dezakegu.

8. PROGRAMAZIOA

- **Main_Activity** beste klase batetik hedatzen da(AppCompatActivity). Klaseek beste klase bati buruz dugun ezagutza guztia har dezakete, eta haien alde erabili, guraso-klasearen kode osoa errepikatu behar ez izateko; **hedatu** esaten zaio, eta objektuetara bideratutako programazioan oso maiz gertatuko den zerbait izango da.

- Bestalde, hemen funtzio bat dugu (**fun** hitz erreserbatua), funtzio horiek askatasunez egon daitezke eta ez dira inorenak edo klase batekoak izan daitezke guk klase baten barruan funtzioak definitzen ditugunean.
- Klase bat zabaltzen dugunean eta aita klaseko funtzio batek egiten duena aldatu nahi dugunean, **override** hitz erreserbatua erabiltzen dugu, eta horrek esan nahi du aita klasea funtzio horretarako erabiltzen ari zen kodea berridatziko dugula.

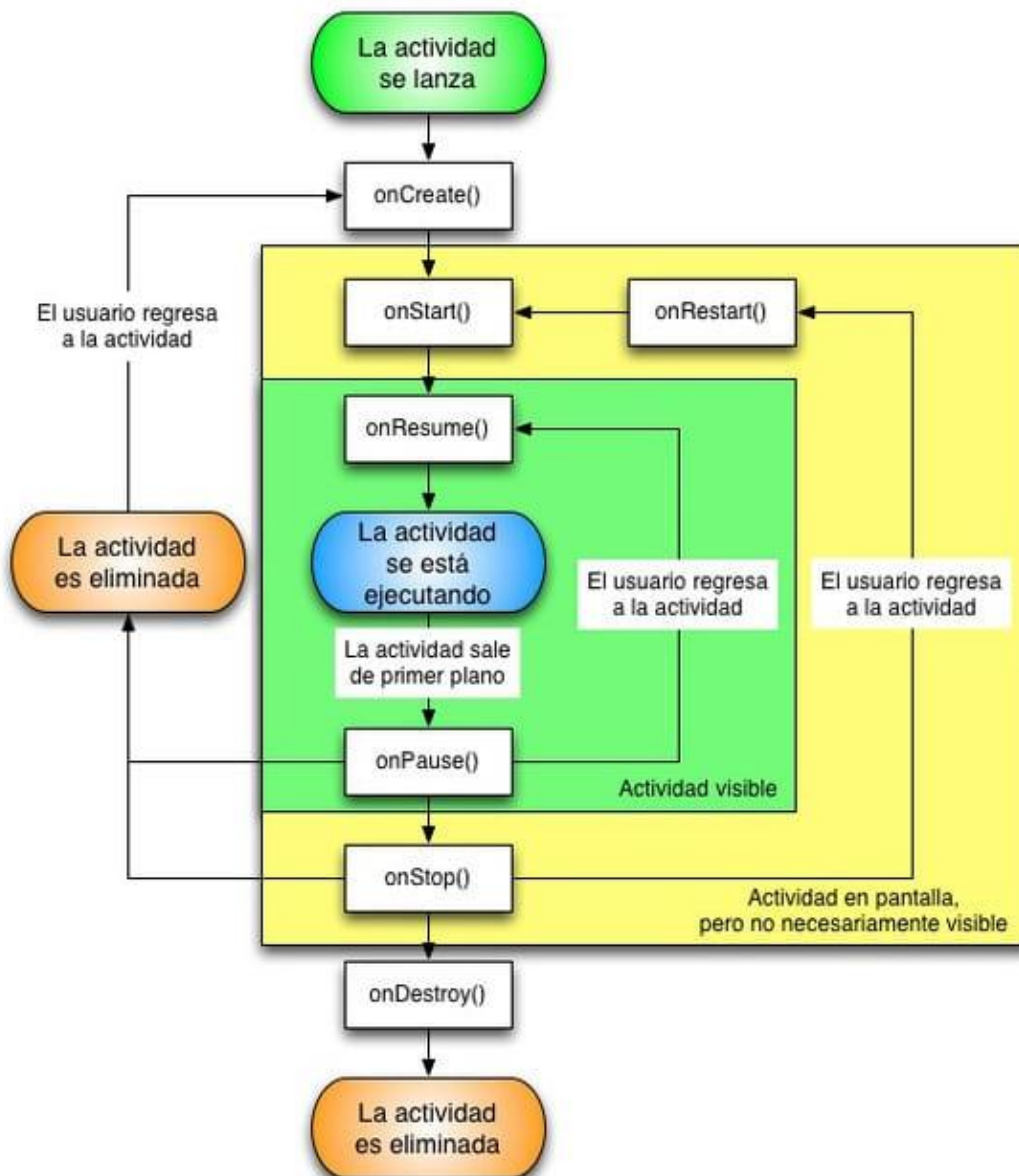
```
class MainActivity : AppCompatActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        enableEdgeToEdge()  
        setContentView(R.layout.activity_main)  
        val p1 = Persona( izena = "Ane", adin = 22)  
        p1.agurtu( context = this)  
        /*setContent {  
            HelloWorldTheme {  
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->  
                    Greeting(  
                        name = "Android",  
                        modifier = Modifier.padding(innerPadding)  
                    )  
                }  
            }  
        }*/  
    }  
}
```

setContentView(R.layout.activity_main)

- Honek xml-a puztea esaten zaiona egingo du: xml testu-fitxategi bat besterik ez da, xml horretan erabiltzen ari garen bistei dagozkien klaseetako objektu serie bat bihurtzea da, guk gero kode bidez bista horietara sartu ahal izateko.
- Xml batetik bista bat berreskuratzeke egin behar dugun gauza bakarra findViewById funtzioa erabiltzea da, eta bista honetan jarri diogun R identifikatutik pasatzea.
- R Android klase berezi bat da, Android-eko baliabideen karpetaaren azpian biltzen diren baliabide guztiak batzen dituen.

```
class MainActivity : ComponentActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        // enableEdgeToEdge()  
        setContentView(R.layout.activity_main)  
        val hellotext=findViewById<TextView>(id = R.id.txIzenburua);  
        hellotext.text="Hello world";  
        // TODO: Replace the following line with your app logic  
    }  
}
```

9. ACTIVITY BATEN BIZI-ZIKLOA



1. onCreate()

- **Noiz deitzen da:** Activity lehen aldiz sortzen denean.
- **Zer egin behar da:**
 - Layouta kargatu ([setContentView\(\)](#)).
 - Konponenteak (botoiak, testuak...) aurkitu eta hasieratu.
 - Datuak edo baliabideak prestatu.
- **Praktika onak:**
 - Ez kargatu hemen logika astunik edo datu handirik (erabili [ViewModel](#) edo [Coroutine](#) asinkronoak).
 - Gorde [Bundle](#) [savedInstanceState](#) erabiltzen bada egoera berreskuratzeko da.

2. onStart()

- **Noiz deitzen da:** Activity pantailan ikusgai jartzen denean (baina ez oraindik interaktibo).
- **Zer egin behar da:**
 - UI eguneratu behar bada, hemen egin.
 - Behaketa (listener) batzuk has daitezke (adib. [location updates](#) edo [LiveData.observe\(\)](#)).
- **Praktika onak:**
 - Ez gehiegi kargatu logika hemen, erabiltzailea oraindik ez dagoelako interakzioan.

3. onResume()

- **Noiz deitzen da:** Activity erabiltzailearentzat aktibo eta interaktibo bihurtzen denean.
- **Zer egin behar da:**
 - Animazioak edo sarrera-interfazeak berrabiarazi.
 - Erabiltzailearen ekintzak onartu (inputa, botoiak...).
- **Praktika onak:**
 - Hemendik aurrera aplikazioa “funtzionamenduan” dago.

4. onPause()

- **Noiz deitzen da:** Activity beste baten atzean geratzen denean (beste aplikazio edo pantaila bat irekitzean).
- **Zer egin behar da:**
 - Baliabide astunak askatu.
 - Egunerokoak edo datu garrantzitsuak gorde.
- **Praktika onak:**
 - *Quick save* egin: adibidez, erabiltzaileak testu bat idatzi badu, gorde *SharedPreferences*-etan.
 - Ez egin hemen prozesu luzeak (oso azkar exekutatu behar da).

5. onStop()

- **Noiz deitzen da:** Activity guztiz ikusgaitik desagertzen denean.
- **Zer egin behar da:**
 - Erreprodukzioak gelditu, baliabideak askatu.
 - Hemen gorde datu iraunkorrak (adib. datu-basea, fitxategiak...).
- **Praktika onak:**
 - Ez utzi baliabiderik memorian (leak-ak saihesteko).
 - Log-ak edo telemetriak bidali nahi badira, aukera egokia da.

6. onDestroy()

- **Noiz deitzen da:** Activity behin betiko amaitzen denean edo sistema baliabideak berreskuratzeko kentzen duenean.
- **Zer egin behar da:**
 - Baliabide guztiak garbitu (thread, observer, listener...).
- **Praktika onak:**
 - Ez fidatu beti deituko denik (batzuetan sistema hil dezake activity-a ohartarazi gabe).

7. onRestart()

- **Noiz deitzen da:** Activity onStop()-etik berriro aktibatzen denean.
- **Zer egin behar da:**
 - Eguneratu interfazea edo berrabiarazi behar diren datuak.
- **Praktika onak:**
 - Ez da beti beharrezkoa, baina UI berriz freskatu nahi bada, aproposa da.

Laburpena

- **onCreate():** Hasieratzea, Activity sortzen da.
- **onStart():** Ikusgarria bihurtzen da, pantailan baina ez aktibo
- **onResume():** Aktiboa eta interaktiboa, erabiltzaileak erabiltzen du
- **onPause():** Bigarren planoan sartzen da, oraindik ikusgarria izan daiteke
- **onStop():** Ikusgaitik kentzen da, ez da pantailan agertzen
- **onDestroy():** Amaiera, activity ezabatzen da
- **onRestart():** Berraktibazioa, onStart() aurretik deitzen da

Praktika onak!

1. **Erabili ViewModel-ak eta LiveData datuak mantentzeko.**
2. **Ez gorde logika astunik Activity-n:** erabili Repository edo Coroutine.
3. **onPause():** gorde egoera bizkor.
4. **onStop():** gorde datu iraunkorrak.
5. **onDestroy():** garbitu listener eta baliabide guztiak (memory leak saihesteko).
6. **LifecycleObserver** erabili daiteke kode garbiagoa izateko.